ADR_013 取消與付款的競爭：不互斥，以自動退款補救

> 狀態：已接受
> 日期：2026-10-15

背景

取消結帳單與發起付款，各自先檢查再寫入，兩者之間沒有共同的鎖（《設計_002_狀態圖與循序圖》3.2、3.7）：

時間	取消（會員主動取消或逾時取消任務）	發起付款
1	向付款模組確認：沒有成功或處理中的付款	
2		確認結帳單為待付款且未逾期，建立處理中的付款，向金流請款
3	結帳單改為已取消，釋放預扣的庫存	
4		金流回報付款成功

結果是結帳單已取消、庫存已釋放，會員的錢卻已被扣款，也就是使用者故事 P4 的情況。

這種情況需要取消與付款在數毫秒內同時發生，例如會員在兩個分頁分別操作，或付款請求剛好落在付款期限的邊界，發生的機會很小。但牽涉到會員的款項，必須明確決定如何處理。

主應用程式現階段是模組化單體，但預計之後拆成微服務，訂單與付款屆時各有自己的資料庫（《設計_003_系統架構》第 7 節）。

考慮過的選項

選項	做法	優點	缺點
以結帳單的列鎖互斥	發起付款與取消都先以 `SELECT ... FOR UPDATE` 鎖定結帳單，在同一個交易內完成檢查與寫入	結果確定，只會是取消或付款其中之一；實作簡單	建立付款必須與訂單模組在同一個交易內；拆成服務後無法成立，必須重寫；須確保持有鎖時不呼叫金流
訂單模組記錄「付款進行中」	發起付款前以條件更新把結帳單標為付款中，取消的條件為「不是付款中」；付款失敗時以事件清除標記	不需要跨模組的交易，拆成服務後仍可使用	結帳單多一個狀態，狀態圖與每個讀取結帳單狀態的地方都要配合；清除標記的事件延遲期間，會員無法取消也無法重新付款；複雜度明顯較高
接受競爭，以自動退款補救	不讓兩者互斥；付款成功時若結帳單已取消，自動全額退款	不增加模組間的耦合，拆成服務時不需要修改；補救機制本來就必須存在	會員會短暫被扣款；帳本多一組收款與退款的分錄；第一版還沒有退款的介面，需人工處理

決定

採用「接受競爭，以自動退款補救」：

- 取消與發起付款不互斥，維持《設計_002_狀態圖與循序圖》3.2、3.7 的流程
- 兩者同時發生的結果只會是以下三種之一，都視為正確：
- 取消被拒絕，付款繼續進行
- 付款被拒絕，結帳單已取消
- 兩者都成功：付款之後若成功，結帳單維持已取消，訂單模組發布「要求自動退款」事件，由付款模組全額退款
- 第一版還沒有退款的介面，付款模組收到「要求自動退款」事件時只發出告警，由平台營運處理（《設計_005_API規格》4.1）
- 以併發測試驗證結果一定落在上述三種之內，且庫存不會同時處於「已釋放」與「已確認扣除」（《設計_006_測試計畫》6.1）

理由：

- 跨服務的競爭無法以資料庫的鎖消除，分散式系統的標準做法是允許它發生，再以補償處理。現階段就依賴補償，拆成服務時這部分的行為不需要改變
- 自動退款不是為了這個競爭才新增的機制。對帳把失敗更正為成功、金流重複成功，同樣會走到這條路，不論採用哪個選項都必須存在
- 以列鎖互斥，等於做一次之後一定要拆掉的設計

後果

- 好處：訂單與付款之間沒有新增的交易耦合；處理 P4 的路徑在第一版就會被測試與使用，不會只是紙上的設計
- 代價：會員在少數情況下會先被扣款再退款；退款階段完成之前需要人工退款；併發測試只能驗證結果落在允許的範圍內，不能斷言唯一的結果
- 重新檢討的條件，符合任一項即重新評估並新增 ADR：
- 營運紀錄或壓測顯示這種競爭的發生次數不可忽略
- 退款階段完成之前，人工退款的件數造成營運負擔
- 需求改為要求取消與付款必須互斥，不接受先扣款再退款